LAB 01

# Programas de usuário

*hello, sleep e pingpong: executar, observar e analisar*

| | |
|---|---|
| **Aluno(a)** | _____ Data: ____/____/_____ |

## ETAPA 1 /7 Preparar

Abra o ambiente:

```
bash xv6.sh
```

## ETAPA 2 /7 Instalar os programas (copiar e colar)

Copie cada bloco INTEIRO (da linha cat até a linha EOF) e cole no terminal (xv6). Cada bloco cria um arquivo em user/.

### 1) user/hello.c

```
cat > user/hello.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("Ola! Eu sou um processo do xv6. Meu PID e %d\n", getpid());
  exit(0);
}
EOF
```

### 2) user/tempo.c

```
cat > user/tempo.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
```

```
  printf("ticks desde o boot: %d\n", uptime());
  exit(0);
}
EOF
```

### 3) user/sleep.c

```
cat > user/sleep.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc < 2){
    fprintf(2, "uso: sleep <ticks>\n");
    exit(1);
  }
  int ticks = atoi(argv[1]);
  sleep(ticks);
  exit(0);
}
EOF
```

### 4) user/pingpong.c

```
cat > user/pingpong.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int pai_filho[2]; // pai escreve, filho le
  int filho_pai[2]; // filho escreve, pai le
  char byte = 'x';

  pipe(pai_filho);
  pipe(filho_pai);

  int pid = fork();
  if(pid == 0){
    // FILHO
    close(pai_filho[1]);
    close(filho_pai[0]);
    read(pai_filho[0], &byte, 1);
    printf("%d: received ping\n", getpid());
    write(filho_pai[1], &byte, 1);
    close(pai_filho[0]);
    close(filho_pai[1]);
    exit(0);
  } else {
    // PAI
    close(pai_filho[0]);
```

```
    close(filho_pai[1]);
    write(pai_filho[1], &byte, 1);
    read(filho_pai[0], &byte, 1);
    printf("%d: received pong\n", getpid());
    close(pai_filho[1]);
    close(filho_pai[0]);
    wait(0);
    exit(0);
  }
}
EOF
```

### 5) Registrar os 4 programas no Makefile (lista UPROGS)

```
grep -q _hello Makefile || sed -i '/_zombie/a\\t$U/_hello\\' Makefile
grep -q _tempo Makefile || sed -i '/_zombie/a\\t$U/_tempo\\' Makefile
grep -q _sleep Makefile || sed -i '/_zombie/a\\t$U/_sleep\\' Makefile
grep -q _pingpong Makefile || sed -i '/_zombie/a\\t$U/_pingpong\\' Makefile
```

### 6) Compilar e ligar

```
make qemu
```

✓*Pronto quando: aparece o prompt $.*

> **Dica**
> *Se aparecer "exec hello failed", o passo 5 não foi feito: Ctrl-a, x, cole o passo 5 e rode make qemu de novo.*

## ETAPA 3 /7 hello

No xv6, rode 3 vezes:

```
$ hello
```

| Execução | PID mostrado |
|---|---|
| 1ª | 3 |
| 2ª | 4 |
| 3ª | 5 |

> **SUAS OBSERVAÇÕES**
> Por que o PID muda a cada execução?

O PID muda porque cada execução do *hello* gera um novo processo e o sistema operacional atribui um

Identificador (PID) diferente a ele.

## ETAPA 4 /7 sleep

| Comando no xv6 | Resultado |
|---|---|
| `sleep` | uso: sleep <ticks> |
| `sleep 10` | - |
| `tempo; sleep 50; tempo` | ticks desde o boot: 2095<br>ticks desde o boot: 2145 |

✍ **SUAS OBSERVAÇÕES**

a) No código do sleep.c (bloco 3), qual linha faz a system call?

b) Quantos ticks passaram entre os dois tempo? Quanto tempo foi no relógio?

a) `sleep(ticks).`

b) Passaram-se 50 ticks. (2145 – 2095). Como no xv6 padrão cada tick equivale a 0,1 segundos, o tempo total foi

de 50 segundos (50 x 0,1) .

## ETAPA 5 /7 pingpong

```
$ pingpong
```

| Execução | PID ping (filho) | PID pong (pai) |
|---|---|---|
| 1ª | 14 | 13 |
| 2ª | 17 | 16 |

✍ **SUAS OBSERVAÇÕES**

a) Qual PID é maior, o do pai ou o do filho? Por quê?

b) No código (bloco 4), o que garante que “ping” aparece antes de “pong”?

a) O PID do filho é maior nas duas execuções.

b) A sincronização por pipes, principalmente as operações *read() e write()*. O filho recebe a mensagem, imprimi *ping* e depois envia uma resposta ao pai. O pai fica bloqueado no *read()* até receber essa reposta, imprimindo o *pong* somente depois.

## ETAPA 6 /7 Experimentos: ppa e ppb

Desligue o xv6 (Ctrl-a, x) e cole os comandos abaixo. Eles criam cópias do pingpong com uma linha desligada (//) e registram as cópias:

```
sed 's|write(filho_pai\[1\]|// &|' user/pingpong.c > user/ppa.c
sed 's|close(filho_pai\[1\]);|// &|' user/ppa.c > user/ppb.c
grep -q _ppa Makefile || sed -i '/_zombie/a\\t$U/_ppa\\' Makefile
grep -q _ppb Makefile || sed -i '/_zombie/a\\t$U/_ppb\\' Makefile
```

| Programa | O que foi desligado |
|---|---|
| ppa | o FILHO não devolve o byte (write) |
| ppb | igual ao ppa + o PAI não fecha filho_pai[1] (close) |

Rode make qemu. Preveja e depois rode no xv6 (se travar: Ctrl-a, x):

| Comando | Previsão | Observado |
|---|---|---|
| ppa | 4 | 3 |
| ppb | 6 | - |

✍ **SUAS OBSERVAÇÕES**

Por que o ppa termina e o ppb trava? (Quando o read devolve 0?)

Em ***ppa*** termina porque todas as extremidades de escrita do pipe são fechadas, fazendo *read()* retornar 0 (EOF). O ***ppb*** trava porque o pai mantém uma extremidade de escrita aberta enquanto espera dados que nunca serão enviados.

## ETAPA 7 /7 Testes automáticos

Com o xv6 desligado (Ctrl-a, x):

```
./grade-lab-util sleep
./grade-lab-util pingpong
```

| Teste | OK / FAIL |
| --- | --- |
| `sleep` | Ok |
| `pingpong` | Ok |